Lab 12 — ¿Por qué 8 núcleos no son 8× más rápido?

Computación Paralela y Distribuida (CC3069) · Sep 30, 2026 · @Luis

Jonathan Diaz 23837

David Dominguez 23712

https://github.com/DavidDominguez-11/Lab-SpeedUp

Objetivo

Vas a medir cuánto más rápido corre un programa al agregar hilos y vas a descubrir, con tus propios datos, por qué casi nunca llega a ser p veces más rápido. Ya viste en clase speedup, eficiencia, las cuatro fugas, la suma en dos fases y la efectividad (hasta la diapositiva 18): aquí los pones a prueba en tu máquina. Lo que sigue —fracción paralelizable, Amdahl, Gustafson, speedup superlineal y cómo medir bien— lo vas a descubrir con tus datos antes de verlo en clase.

Al terminar podrás:

Medir tiempos de ejecución de forma confiable con omp_get_wtime().

Calcular speedup, eficiencia, costo y efectividad a partir de mediciones reales.

Identificar qué limita el speedup: la parte secuencial, el overhead, el desbalance de carga.

Distinguir entre escalar con un problema fijo y escalar con un problema que crece.

Modalidad: parejas · Duración estimada: 2 horas · Lenguaje: C con OpenMP · Entregable: reporte PDF + CSVs + gráficas (ver al final). Regla del lab: antes de cada experimento escribe tu predicción en el reporte. Se califica que la hayas hecho y que expliques la diferencia con lo medido, no que acertaras.

Preparación (15 min)

Descarga lab12_starter.zip del portal. Contiene lab12_speedup.c (los 4 experimentos), un Makefile y graficar.py (calcula S, E, C y F y genera las gráficas).

Sistema	Instalar	Compilar
Linux / WSL	sudo apt install build-essential python3-matplotlib	make

macOS (clang)	brew install libomp y pip3 install matplotlib	make
macOS (gcc)	brew install gcc	make CC=gcc-14 (o la versión que instaló)
Windows sin WSL	Instala WSL: wsl --install	make dentro de WSL

Verifica que funciona con ./lab12 (sin argumentos debe mostrar el uso).

Anota en tu reporte los datos de tu equipo — sin esto tus resultados no se pueden interpretar:

1. Modelo de CPU y cuántos núcleos físicos y lógicos tiene (lscpu en Linux/WSL, sysctl -n machdep.cpu.brand_string hw.physicalcpu hw.logicalcpu en macOS).

2. Si tu CPU tiene núcleos de rendimiento y de eficiencia (Apple M-series, Intel 12.ª gen en adelante), cuántos de cada uno.

3. Si tiene Hyper-Threading / SMT.

Equipo de las mediciones: AMD Ryzen 5 4600H with Radeon Graphics, 6 núcleos físicos, 12 procesadores lógicos (2 hilos por núcleo); arquitectura homogénea, sin núcleos P/E. Tiene SMT; el nombre comercial Hyper-Threading corresponde a Intel. Entorno: Ubuntu 24.04 en WSL2, GCC 13.3.0, OpenMP, pmax = 12. Fuente: salida íntegra de lscpu en resultados_lab12.zip. No se registró el estado de alimentación ni las aplicaciones abiertas, por lo que no se atribuyen las variaciones a esas condiciones.

Para medir bien: laptop conectada a la corriente, navegador y apps cerradas, y no uses la computadora mientras corre un experimento. El programa repite cada medición 5 veces y reporta la mediana.

Repaso: lo que ya viste en clase

Usamos la misma notación de clase. Ts = tiempo de la versión 100% secuencial (sin OpenMP); Tp = tiempo con p hilos.

Medida	Fórmula	Qué responde
Speedup	S = Ts / Tp	¿cuántas veces más rápido?
Eficiencia	E = S / p	¿qué fracción de cada hilo se aprovechó?
Costo	C = p · Tp	¿cuántos hilo-segundos pagué?
Efectividad	F = S / C	¿cuánta rapidez obtengo por unidad de costo?

Y las cuatro fugas de eficiencia: η_f parte secuencial, η_c comunicación y sincronización, η_b desbalance, η_r trabajo redundante. En cada parte vas a identificar cuál estás viendo.

El programa mide; tú calculas. graficar.py también calcula S, E, C y F para que verifiques: en el reporte muestra el cálculo a mano de al menos dos filas.

Cálculos manuales de dos filas: Amdahl fp=0.90, p=4: S=0.646815/0.212111=3.0494; E=3.0494/4=0.7624; C=4×0.212111=0.848444 hilo·s; F=3.0494/0.848444=3.5941 $(\text{hilo}\cdot\text{s})^{-1}$. Suma N=65536, p=4: S=0.000052912/0.000015200=3.4811; E=3.4811/4=0.8703; C=4×0.000015200=0.000060800 hilo·s; F=3.4811/0.000060800=57254.2 $(\text{hilo}\cdot\text{s})^{-1}$. Los cálculos usan los tiempos del CSV original, antes de redondear las métricas exportadas.

Cada experimento imprime un CSV con estas columnas:

Columna	Significado
p	número de hilos usados
t_seq	Ts: versión sin OpenMP del mismo trabajo (s)
t_par	Tp: versión OpenMP con p hilos (s)
t_parte_seq	dentro de Tp, lo que tardó la parte que no se puede repartir (s)
param	el parámetro del experimento (fracción paralelizable fp, schedule, etc.)

Parte 1 — La parte que no se puede repartir (35 min)

El modo amdahl ejecuta W unidades de trabajo. Una fracción fp (fracción paralelizable) se reparte entre p hilos con #pragma omp parallel for ; el resto, 1 − fp, es una cadena donde cada paso depende del anterior. Es la fuga η_f en estado puro. El tamaño del problema no cambia al agregar hilos.

Predicción (antes de correr): con tu número máximo de hilos, ¿qué speedup esperas para fp = 1.00? ¿y para fp = 0.90? Escríbelo.

Predicción registrada antes de medir: para pmax=12, S ideal = 12 con fp=1.00 y S = 1/(0.10+0.90/12) = 5.714 con fp=0.90. Esperaba valores menores por sobrecarga, SMT y frecuencia variable. Se midieron 9.516 y 5.147, respectivamente.

```
./lab12 amdahl 400000 1.00 > amdahl_fp100.csv
./lab12 amdahl 400000 0.95 > amdahl_fp95.csv
./lab12 amdahl 400000 0.90 > amdahl_fp90.csv
./lab12 amdahl 400000 0.75 > amdahl_fp75.csv
./lab12 amdahl 400000 0.50 > amdahl_fp50.csv
python3 graficar.py amdahl_*.csv
```

Llena esta tabla en tu reporte (pmax = tu número de hilos):

fp	S (p=2)	S (p=4)	S (pmax)	E (pmax)	p con F máxima
1	1.985	3.639	9.516	0.793	12
0.95	1.846	3.370	6.270	0.522	8
0.9	1.840	3.049	5.147	0.429	8
0.75	1.694	2.403	2.698	0.225	3
0.5	1.272	1.561	1.824	0.152	1

Preguntas

1. Con fp = 1.00 ¿llegaste a S = p? Si no, propón al menos dos causas pensando en tu hardware (núcleos de eficiencia, Hyper-Threading, turbo boost cuando corre un solo hilo, costo de crear hilos). ¿Alguna encaja en las cuatro fugas?

No: con fp=1.00 y p=12, S=9.516<12 y E=0.793. Dos causas plausibles son compartir cada núcleo físico entre dos hilos SMT y el costo de iniciar/sincronizar el equipo OpenMP; también puede variar la frecuencia del procesador. La sincronización entra en η_c; SMT y frecuencia no quedan descritos por las cuatro fugas. Son explicaciones plausibles, no causas aisladas experimentalmente.

2. Describe la forma de la curva de S cuando fp = 0.75. ¿Qué pasa con la ganancia de agregar el hilo número 8 comparada con la del hilo 2?

La curva crece con rendimientos decrecientes y termina cayendo: S(1)=0.999, S(2)=1.694, S(7)=2.962, S(8)=3.103, máximo medido S(9)=3.200, S(12)=2.698. El segundo hilo aporta ΔS=0.695; el octavo, solo ΔS=0.141.

3. Deriva tu propio modelo. Un programa tarda T1 segundos en un núcleo; una fracción fp de ese tiempo se puede repartir entre p hilos y el resto no. Escribe Tp en función de T1, fp y p. Luego escribe S(p) = T1 / Tp y simplifica para que T1 desaparezca.

Si T_1 se descompone en $(1-fp)T_1$ secuencial y $fp \cdot T_1$ paralelizable, el modelo ideal da $T_p=(1-fp)T_1+(fp \cdot T_1)/p=T_1[(1-fp)+fp/p]$. Por tanto $S(p)=T_1/T_p=1/[(1-fp)+fp/p]$. Supone reparto perfecto y costo de coordinación nulo.

4. Usa tu fórmula para predecir S(pmax) con fp = 0.90 y compárala con lo medido. ¿Qué tan cerca quedó?

Con fp=0.90 y p=12, S ideal=5.714; el CSV da S=0.646815/0.125664=5.147. Quedó 0.567 unidades (9.9%) por debajo del modelo; éste omite overhead y efectos del hardware.

5. ¿A qué valor tiende tu fórmula cuando p → ∞? Con fp = 0.90, ¿cuál es el speedup máximo posible aunque tuvieras 1000 núcleos?

Cuando p→∞, fp/p→0 y S→1/(1−fp). Con fp=0.90 el límite es 10; con 1000 núcleos, S=1/(0.10+0.90/1000)=9.911, todavía menor que 10.

6. Al revés: despeja fp de tu fórmula y calcúlalo con el S que mediste para fp = 0.90 y p = pmax. ¿Te da menos de 0.90? ¿Qué fuga representa esa diferencia?

De 1/S=1−fp+fp/p se obtiene fp=(1−1/S)/(1−1/p). Con S=5.1472 y p=12, fp aparente=0.8790<0.90. La diferencia puede reflejar η_c (coordinación) y otras pérdidas como SMT o frecuencia. No mide una nueva fracción secuencial física del código.

7. Efectividad: para fp = 0.90, ¿con cuántos hilos es máxima F? ¿Es el mismo p que da el menor tiempo? Compáralo con el caso de 16 números de clase, donde el óptimo fue p = 4.

Para fp=0.90, F es máxima con p=8 (F=4.228 (hilo·s)$^{-1}$); el menor tiempo ocurre con p=12 (Tp=0.125664 s). Son objetivos distintos: rapidez frente a rapidez por costo. El ejemplo teórico de 16 números tenía óptimo p=4; aquí el tamaño y la sobrecarga cambian ese óptimo.

8. Un programa tarda 20 s y 2 s de ellos son leer un archivo de forma secuencial. Tu jefe quiere comprar un servidor de 64 núcleos. ¿Qué le respondes, con números de speedup y de efectividad?

Los 2 s secuenciales dejan 18 s paralelizables: T_{64} ideal=2+18/64=2.28125 s, S_{64}=20/2.28125=8.767 y E_{64}=0.137. C_{64}=64×2.28125=146 hilo·s; F_{64}=8.767/146=0.0600 (hilo·s)$^{-1}$. A p=9, el mismo modelo da T_9=4 s, S_9=5, C_9=36 y F_9=0.1389: 64 núcleos reducen latencia, pero con mucha menor efectividad que 9. Además son límites optimistas, antes de overhead.

Parte 2 — ¿Vale la pena paralelizar algo pequeño? (25 min)

2a. El Ts honesto. Vuelve a amdahl_fp100.csv y compara t_seq con t_par en la fila p = 1. Son el mismo trabajo: uno sin OpenMP y otro con OpenMP usando un solo hilo.

1. ¿Cuál es más rápido y por cuánto (en %)?

En la corrida fp=1.00, la versión OpenMP con un hilo fue ligeramente más rápida: Ts=0.596371 s, Tp(1)=0.595303 s. La diferencia relativa es 0.18% respecto de Ts, pequeña y compatible con variación de medición.

2. Si alguien reporta su speedup usando como Ts el programa paralelo con 1 hilo, ¿su speedup sale inflado o desinflado? ¿Por qué es tramposo?

Aquí saldría ligeramente desinflado ya que usar Tp(1)=0.595303 en vez de Ts=0.596371 reduce cualquier S calculado por un factor 0.9982. No debe asumirse siempre la dirección: si Tp(1)>Ts, saldría inflado. Es tramposo cambiar la línea base porque Ts está definido como la versión sin OpenMP del mismo trabajo.

2b. Sumar N números. El modo suma suma N números con un reduction(+:s) , para N desde 16 hasta 16.7 millones. Para que los tiempos sean medibles, cada suma se repite muchas veces y se reporta el tiempo de una suma.

Predicción: ¿a partir de qué N crees que la versión paralela empieza a ganarle a la secuencial?

Predicción previa: para N=16 y 1024 la sobrecarga dominaría; estimé cruce alrededor de N=65536 o 1048576. En los tamaños probados, el primer S>1 con p≥2 apareció en N=65536; por ejemplo, con p=4, S=3.481.

```
./lab12 suma > suma.csv
python3 graficar.py suma.csv
```

Preguntas

3. Con N = 16 y p = pmax, ¿cuánto vale S? Exprésalo como "la versión paralela es X veces más lenta".

Para N=16 y pmax=12: S=0.000000004/0.000005135=0.000779; Tp/Ts=1283.8. En este ensayo, la suma OpenMP fue aproximadamente 1284 veces más lenta. Ts se redondea a nanosegundos en el CSV; la razón es aproximada.

4. En clase, con el modelo de dos fases, 16 números y p = 4 daban Tp = (16/4 − 1) + log2 4 = 5 sumas y S = 3. ¿Qué supuesto de ese modelo no se cumple en tu máquina? ¿Qué fuga es?

El modelo de 5 pasos solo cuenta sumas y supone costo cero de crear/despertar hilos, distribuir trabajo, combinar parciales y esperar. En el equipo, con N=16 y p=4, S=0.0034,

muy por debajo de 3. Domina η_c, la sobrecarga de coordinación frente a apenas 16 datos.

5. Lista lo que OpenMP tiene que hacer en cada parallel for además de sumar (despertar hilos, repartir, combinar el reduction , esperar en la barrera…). ¿En qué se va el tiempo con N = 16?

Cada parallel for debe entrar a la región, disponer los hilos, asignar iteraciones, crear acumuladores privados, combinar el reduction y sincronizar al salir. Con N=16, estas operaciones duran mucho más que las 16 sumas; Tp(12)=5.135 μs frente a Ts≈0.004 μs por suma.

6. Estima a partir de qué N la versión paralela empieza a convenir en tu equipo. ¿Qué regla práctica sacas sobre cuándo vale la pena un parallel for ?

Entre los cinco tamaños medidos, N=65536 es el primero donde p=2,4,8 y 12 ganan a la versión secuencial (con p=12, S=4.276); en N=1024, p=12 da S=0.190. El umbral real está entre 1024 y 65536 para esta configuración; conviene que cada región paralela haga suficiente trabajo para amortizar su costo, y medir el cruce para cada equipo.

7. Con N grande, ¿llega S a p? La suma hace muy poco cálculo por cada dato que lee de memoria; ¿qué recurso compartido entre todos los núcleos podría estar limitando? ¿Encaja en alguna de las cuatro fugas?

Con N=16777216 y p=12, S=3.095<12 y E=0.258; incluso baja frente a N=1048576. El ancho de banda de memoria compartida es una explicación plausible porque se lee mucho por cada operación. No se midieron contadores de memoria, así que no se puede demostrar la causa; una limitación de ancho de banda no aparece de forma explícita entre η_f, η_c, η_b y η_r.

Parte 3 — Cuando a un hilo le toca más trabajo (20 min)

El modo desbalance tiene un ciclo donde la iteración i cuesta proporcional a i: las primeras son baratas y las últimas caras. Es la fuga η_b (desbalance) de la clase, ahora medida. El trabajo total es el mismo; lo que cambia es cómo se reparten las iteraciones (schedule).

- static : se parte el rango en p bloques contiguos iguales antes de empezar.
- dynamic : cada hilo toma bloques de 64 iteraciones a medida que se desocupa.
- guided : como dynamic, pero los bloques empiezan grandes y se van achicando.

Predicción: con p = 4 y static , ¿qué fracción del trabajo total le toca al último hilo? ¿Qué speedup esperas entonces?

Predicción previa: con p=4 y costo aproximadamente lineal, el último cuarto concentra ≈7/16=43.75% del trabajo, por lo que S ideal≈16/7=2.286. Medido: S_static(4)=2.235, cercano al valor previsto.

```
./lab12 desbalance static > desb_static.csv
./lab12 desbalance dynamic > desb_dynamic.csv
./lab12 desbalance guided > desb_guided.csv
python3 graficar.py desb_*.csv
```

Preguntas

1. Compara S(pmax) de los tres schedules. ¿Cuál gana y por cuánto?

En pmax=12: S_static=3.485, S_dynamic=10.704 y S_guided=10.735. Guided obtuvo el mayor S, apenas 0.29% por encima de dynamic y 3.08 veces el de static. La diferencia guided/dynamic es pequeña para atribuirle superioridad general.

2. Con static , calcula a mano qué fracción del trabajo le toca al hilo más cargado para p = 2 y p = 4 (pista: el costo crece como una recta, el trabajo de un bloque es el área de un trapecio). Con eso predice S y compáralo con lo medido.

Aproximando costo(i)=1+2000·i/W, el total normalizado es $\int_0^1(1+2000x)dx=1001$. Para p=2, el último bloque [1/2,1] recibe (0.5+750)/1001=0.7498; S ideal≈1/0.7498=1.334, frente a S medido=1.304. Para p=4, [3/4,1] recibe (0.25+437.5)/1001=0.4373; S ideal≈2.287, frente a 2.235. La fórmula discreta real difiere ligeramente por el redondeo entero de costo.

3. En static , ¿qué están haciendo los primeros hilos mientras el último termina? ¿Cómo se ve eso en la eficiencia?

Los hilos con bloques baratos terminan pronto y esperan al último en la barrera. La eficiencia cae a E_static(4)=0.559 y E_static(12)=0.290; a p=12, guided llega a E=0.895. La caída de static en p=12 también puede contener variación del sistema, además del desbalance previsto.

4. Si dynamic es mejor aquí, ¿por qué no usarlo siempre? Piensa qué costo extra tiene repartir en tiempo de ejecución y relácionalo con lo que viste en la Parte 2.

Dynamic requiere asignar bloques en tiempo de ejecución y sincronizar las solicitudes; con tareas diminutas ese costo puede superar la ventaja de balancear. La Parte 2 mostró que la coordinación dominó al sumar solo 16 valores. Aquí las iteraciones son suficientemente costosas para que dynamic y guided compensen ese overhead.

5. Si tu CPU tiene núcleos de rendimiento y de eficiencia, ¿por qué un static con iteraciones de costo idéntico también podría quedar desbalanceado?

En una CPU híbrida, núcleos de rendimiento y de eficiencia procesan cantidades iguales de iteraciones a velocidades distintas; static puede dejar esperando a los rápidos. Este Ryzen 5 4600H no tiene esa mezcla P/E, por lo que es una explicación hipotética y no una observación de esta corrida.

Parte 4 — ¿Y si el problema crece con los núcleos? (25 min)

En la Parte 1 el problema era fijo. Ahora el modo gustafson mantiene fija la parte secuencial (20 000 unidades) pero le da a cada hilo 100 000 unidades paralelas: con p hilos el problema total es 20 000 + 100 000·p. t_seq es lo que tardaría ese problema más grande en un solo núcleo.

Predicción: con p = 1 la parte secuencial es 1/6 del trabajo (~17%). Según tu fórmula de la Parte 1, ¿qué speedup máximo tendría un programa con fp = 0.83? ¿Crees que aquí pase lo mismo?

Predicción previa: si fp≈5/6 fuera fijo, Amdahl limitaría S a $1/(1-5/6)=6$ aun con infinitos hilos. Para el problema escalado y p=12, la predicción ideal de Gustafson es $12-(12-1)/6=10.167$. Medimos S(12)=8.914: supera el límite aproximado de 6 para el problema fijo, pero queda por debajo de 10.167.

```
./lab12 gustafson > gustafson.csv
python3 graficar.py gustafson.csv
```

Preguntas

1. ¿Cómo cambian t_par y t_seq conforme crece p? Explica por qué uno casi no se mueve y el otro crece.

Al pasar de p=1 a p=12, Ts crece de 0.198487 a 1.988800 s porque el trabajo paralelo total pasa de 100000 a 1200000 unidades. Tp cambia de 0.200283 a 0.223116 s: cada hilo recibe aproximadamente 100000 unidades, aunque el tiempo real sube por costos y recursos compartidos.

2. Compara S(pmax) con lo que predice tu fórmula de la Parte 1 para fp = 0.83. ¿Por qué aquí el resultado es tan distinto si el código es el mismo?

Aplicar Amdahl a fp=0.83 fijo y p=12 predice S=4.181 (límite cuando p→∞: 5.882); se midió S=8.914. No hay contradicción: en Gustafson, N crece con p y la fracción paralelizable del problema secuencial equivalente también cambia. Amdahl con fp fijo responde a un tamaño de problema fijo.

3. Calcula s = t_parte_seq / t_par para cada fila. ¿Se mantiene o cambia con p?

s=t_parte_seq/t_par por cada p (p:s): 1:0.166, 2:0.170, 3:0.166, 4:0.164, 5:0.169, 6:0.164, 7:0.166, 8:0.163, 9:0.160, 10:0.161, 11:0.160, 12:0.153. Se mantiene cerca de 0.16–0.17, con ligera caída a 0.153 en p=12; no es exactamente constante por variación de tiempos y overhead.

4. Deriva el segundo modelo. Si en la ejecución paralela una fracción s de Tp es secuencial y el resto (1 − s)·Tp lo hacen p hilos en paralelo, ¿cuánto tardaría ese mismo trabajo en un solo núcleo? Escribe S = Ts / Tp en función de s y p. Verifica tu fórmula con tus datos.

En el modelo ideal, Ts_equiv=s·Tp+p(1−s)·Tp. Dividiendo por Tp: $S_G=s+p(1-s)=p-(p-1)s$. Con p=4 y s=0.1645, predice 3.507; se midió 3.318. Con p=12 y s=0.1531, predice 10.316; se midió 8.914. La diferencia creciente indica que escalar 12 hilos no equivale a multiplicar por 12 el rendimiento de uno; también influyen overhead, SMT y frecuencia.

5. Cada experimento responde una pregunta distinta: "¿cuánto más rápido resuelvo el mismo problema?" y "¿qué tanto más grande puedo hacer el problema en el mismo tiempo?". Asocia cada pregunta con la Parte 1 o la Parte 4 y da un ejemplo real de cada una (pronóstico del clima, renderizado, entrenamiento de modelos, una consulta web…).

Parte 1, problema fijo: «¿cuánto antes termino el mismo render de una imagen?». Parte 4, problema escalado: «¿cuántos fotogramas puedo renderizar en un tiempo semejante si añado núcleos?». En el segundo caso aumenta el trabajo total, por lo que se usa un modelo distinto.

Síntesis, reto y entrega

Síntesis (obligatoria). Para cada parte, di qué fuga (η_f, η_c, η_b o η_r) explica la eficiencia que perdiste, con el número medido que lo demuestra. Luego lista lo que no encaja en ninguna de las cuatro (pistas: preguntas 1.1 y 2.7). Tráelo a la clase: es justo lo que vemos después de la efectividad.

Síntesis: (1) η_f: con fp=0.90, E(12)=0.429; η_c y hardware también reducen fp=1.00 a E(12)=0.793. (2) η_c domina la suma N=16: E(12)=0.000065; en N=16777216, E(12)=0.258, compatible además con límite de memoria compartida. (3) η_b: a p=12, static E=0.290 frente a guided E=0.895. (4) η_f persiste: s(12)=0.153; overhead y hardware ayudan a explicar S medido 8.914 frente a S_G ideal 10.316. No se aisló η_r en estos programas. SMT/frecuencia y ancho de banda de memoria no quedan identificados por las cuatro fugas; se citan como hipótesis, no como diagnósticos medidos.

Reto opcional (+10). Consigue un speedup mayor que p (S > p) y explica por qué ocurre, o demuestra que el que obtuviste era falso.

- Pista 1: cada núcleo tiene su propia caché. ¿Qué pasa si el arreglo completo no cabe en la caché de un núcleo, pero cada pedazo sí cabe en la caché de cada núcleo? Puedes cambiar los tamaños de tamanos[] en exp_suma .
- Pista 2: un S > p que desaparece al repetir la medición no cuenta. Reporta al menos 5 corridas.

Entrega: un ZIP con el reporte en PDF, todos los CSV y las gráficas PNG. El reporte incluye los datos de tu equipo, tus predicciones, las tablas y las respuestas numeradas como en esta guía.

Criterio	Puntos
Datos del equipo, CSVs y gráficas completos	15
Predicciones escritas y contrastadas con lo medido	15
Parte 1 — tabla, modelo derivado y preguntas	25
Parte 2 — Ts honesto y granularidad	15
Parte 3 — desbalance de carga	15
Parte 4 — problema que crece y segundo modelo	15
Reto opcional	10